

Razorpay Payment Gateway Integration Guide

Stack: Django (Python) + HTML, CSS, JavaScript

Author: ServeSync Team

Date: June 2026

Table of Contents

1. [Create a Razorpay Account](#)
 2. [Install Razorpay in Django](#)
 3. [Add Keys to Django Settings](#)
 4. [Create Order in Django Backend](#)
 5. [Verify Payment on Backend](#)
 6. [Frontend HTML + CSS + JS](#)
 7. [Test the Payment](#)
 8. [How It All Flows](#)
 9. [Go Live Checklist](#)
-

Step 1 — Create a Razorpay Account

1. Go to **razorpay.com** and click **Sign Up**
2. Enter your business or personal details and verify your email
3. After login, go to **Settings** → **API Keys**
4. Click **Generate Test Mode API Keys**
5. You will get two keys — copy and save them somewhere safe:
 - **Key ID** — starts with `rzp_test_...`
 - **Key Secret** — a long random string

Note: Keep Test Mode ON while building. Switch to Live Mode only when going to production.

Step 2 — Install Razorpay in Django

Run this command in your terminal inside your Django project folder:

```
pip install razorpay
```

Then add it to your `requirements.txt`:

```
razorpay
```

Step 3 — Add Keys to Django Settings

Open your `settings.py` file and add these two lines:

```
RAZORPAY_KEY_ID      = "rzp_test_XXXXXXXXXXXXX"  
RAZORPAY_KEY_SECRET = "your_secret_key_here"
```

Important: Never push real keys to GitHub. For production, store keys in a `.env` file and use the `python-decouple` library to read them.

Step 4 — Create Order in Django Backend

In your `views.py`, add the following code:

```
import razorpay  
import json  
from django.conf import settings  
from django.http import JsonResponse  
from django.views.decorators.csrf import csrf_exempt  
  
client = razorpay.Client(auth=(settings.RAZORPAY_KEY_ID,  
settings.RAZORPAY_KEY_SECRET))  
  
def create_order(request):  
    if request.method == "POST":  
        data = json.loads(request.body)  
        amount = data.get("amount") # amount in rupees  
  
        order = client.order.create({  
            "amount": int(amount) * 100, # Razorpay needs paise (₹1 =  
100 paise)  
            "currency": "INR",  
            "payment_capture": 1  
        })  
  
        return JsonResponse({  
            "order_id": order["id"],  
            "amount": order["amount"],  
            "currency": order["currency"],  
            "key_id": settings.RAZORPAY_KEY_ID,  
        })
```

In your `urls.py`, register the routes:

```
from django.urls import path  
from . import views
```

```
urlpatterns = [
    path("create-order/", views.create_order, name="create_order"),
    path("verify-payment/", views.verify_payment, name="verify_payment"),
]
```

Step 5 — Verify Payment (Backend)

After the customer pays, Razorpay sends back 3 values. You must verify them on the server to confirm the payment is genuine:

```
@csrf_exempt
def verify_payment(request):
    if request.method == "POST":
        data = json.loads(request.body)

        params = {
            "razorpay_order_id": data["razorpay_order_id"],
            "razorpay_payment_id": data["razorpay_payment_id"],
            "razorpay_signature": data["razorpay_signature"],
        }

        try:
            client.utility.verify_payment_signature(params)
            # Payment verified – save the order to your database here
            return JsonResponse({"status": "success"})

        except razorpay.errors.SignatureVerificationError:
            return JsonResponse({"status": "failed"}, status=400)
```

Step 6 — Frontend (HTML + CSS + JS)

Create your checkout HTML page. Razorpay provides a JavaScript library that opens the payment popup automatically:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8" />
    <title>Checkout</title>

    <!-- Load Razorpay JS library -->
    <script src="https://checkout.razorpay.com/v1/checkout.js"></script>

    <style>
        body {
            font-family: Arial, sans-serif;
            display: flex;
```

```
    flex-direction: column;
    align-items: center;
    justify-content: center;
    height: 100vh;
    background: #f5f5f5;
  }

  .card {
    background: white;
    padding: 40px;
    border-radius: 12px;
    box-shadow: 0 4px 20px rgba(0,0,0,0.1);
    text-align: center;
  }

  h2 { color: #333; margin-bottom: 20px; }

  button {
    background: #f97316;
    color: white;
    border: none;
    padding: 14px 32px;
    border-radius: 8px;
    font-size: 16px;
    font-weight: bold;
    cursor: pointer;
  }

  button:hover { background: #ea6a0a; }
</style>
</head>
<body>

<div class="card">
  <h2>Total Amount: ₹<span id="amount">500</span></h2>
  <button onclick="startPayment()">Pay Now</button>
</div>

<script>
  async function startPayment() {
    const amount = document.getElementById("amount").innerText;

    // Step 1: Call Django to create an order
    const response = await fetch("/create-order/", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ amount: amount })
    });
    const order = await response.json();

    // Step 2: Open Razorpay payment popup
    const options = {
      key: order.key_id,
      amount: order.amount,
```

```
currency:    order.currency,
name:        "Your App Name",
description: "Order Payment",
order_id:    order.order_id,

// Step 3: This runs after payment is done
handler: async function(paymentResponse) {
  const verify = await fetch("/verify-payment/", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({
      razorpay_order_id: paymentResponse.razorpay_order_id,
      razorpay_payment_id: paymentResponse.razorpay_payment_id,
      razorpay_signature: paymentResponse.razorpay_signature,
    })
  });

  const result = await verify.json();

  if (result.status === "success") {
    alert("Payment Successful! Thank you.");
    window.location.href = "/orders/";
  } else {
    alert("Payment verification failed. Please contact support.");
  }
},

prefill: {
  name: "Customer Name",
  email: "customer@email.com",
  contact: "9999999999"
},

theme: { color: "#f97316" }
};

const rzp = new Razorpay(options);
rzp.open();
}
</script>

</body>
</html>
```

Step 7 — Test It

Use these **test card details** provided by Razorpay. No real money is charged:

Field	Value
Card Number	4111 1111 1111 1111

Field	Value
Expiry Date	Any future date
CVV	Any 3 digits
OTP	1234

For UPI test: use `success@razorpay`

For failed payment test: use `failure@razorpay`

How It All Flows

```
User clicks "Pay Now"
  ↓
Frontend sends amount to Django → /create-order/
  ↓
Django calls Razorpay API → gets order_id
  ↓
Frontend opens Razorpay popup with order_id
  ↓
User enters card/UPI details and pays
  ↓
Razorpay returns: payment_id + order_id + signature
  ↓
Frontend sends these 3 values to Django → /verify-payment/
  ↓
Django verifies the signature using secret key
  ↓
If valid → save order to database → show success
If invalid → show error
```

Go Live Checklist

Before switching to production, complete these steps:

- ☐ Complete **KYC** on your Razorpay dashboard (business details, bank account)
- ☐ Switch Razorpay dashboard to **Live Mode**
- ☐ Generate **Live API Keys** and replace test keys
- ☐ Move all keys to a **.env file** — never push to GitHub
- ☐ Test with a real **₹1 transaction** to confirm everything works
- ☐ Make sure your website has **HTTPS** (required for live payments)
- ☐ Add proper **error handling** and show user-friendly messages on failure

Common Errors & Fixes

Error	Fix
Invalid key_id	Check you copied the Key ID correctly from Razorpay dashboard
Signature mismatch	Make sure Key Secret in settings.py matches the one in Razorpay
Order not found	The order_id from create_order must be passed to the frontend correctly
Payment popup not opening	Make sure the Razorpay JS script is loaded in <code><head></code>
CSRF error	Add <code>@csrf_exempt</code> decorator to your Django views

Document prepared for educational purposes. Always follow Razorpay's official documentation for production use.